DeepFabric

APS Failure at Scania Trucks - Classification

Round 1 Screener

Problem Statement

You are a Data Analyst at a logistics and fleet-maintenance company that manages thousands of heavy-duty trucks. Unexpected breakdowns cause delivery delays, contractual penalties, towing costs, and safety risks. The maintenance team currently relies on manual checks and rule-based alerts, which leads to unnecessary inspections (false positives) and missed failures (false negatives).

Your manager wants a data-driven predictive maintenance classifier that can flag trucks likely to have a failure in the Air Pressure System (APS) based on historical sensor readings. Your solution should be robust to missing values, scalable to high-dimensional sensor features, and evaluated with metrics suitable for rare failure events.

Dataset

We have attached the required datasets along with this Document, Please find it in your mail

Dataset overview

The dataset contains sensor and operational measurements from Scania trucks. The target indicates whether a failure is related to the APS system or a non-APS failure.

Typical characteristics include:

- Missing values (commonly represented with placeholder strings like `na`)
- Class imbalance (APS failures are relatively rare compared to non-failures)

Target definition

- Positive class: APS failure
- Negative class: non-APS related failure

DeepFabric

Objective

Build a classification solution to predict whether a Scania truck has an APS (Air Pressure System) failure using high-dimensional sensor readings.

Part 1

1. Classification target

- **Predict `class` = APS failure vs non-APS failure**
  - Convert the provided label into a binary target:
    - `1` = APS failure (positive class)
    - `0` = non-APS failure (negative class)
  - Clearly document the mapping you used based on the dataset's label values.

2. Evaluation setup (equivalent of "forecast horizon")

For this classification , evaluate the model under the following operationally relevant evaluation windows:

- **Standard Evaluation :** Use the provided **training** and **test** datasets separately for model training and evaluation. The test dataset includes the ground-truth labels, which will be used to assess your model's performance. **Avoid overfitting to the test set, as the evaluation will emphasize your model's ability to generalize effectively.**
- Threshold-based decisioning: Select an operating threshold for predicted probability such that the model can be used to trigger inspections
- Cost-sensitive perspective (must be discussed):
  - APS failures are rare and expensive to miss → emphasize Recall and PR-AUC.
  - Discuss the operational impact of false positives (unnecessary inspections) vs false negatives (missed failures).

Part 2

Provide the below analytics (you may use appropriate **visualizations** or **NLP-style written insights**).

1. Trend & Data Profiling Overview (Sensor Data EDA)
   - High-level summary of the dataset: rows, columns, target distribution, and any dominant patterns (e.g., constant/near-constant sensors).

2. Missing Data Quality Assessment
    - Missingness per column (top missing features), missingness distribution, and your treatment strategy (drop/impute/indicator flags).
    - Mention how missingness could affect model reliability.
3. Failure Drivers & KPI Summary
    - Identify the most informative sensors/features
    - Include KPIs such as failure rate, high-risk bucket size, and top drivers.
4. Model Performance & Error Diagnostics
    - Confusion matrix at chosen threshold, PR curve, and error analysis (where false negatives/false positives concentrate).
    - Performance summary: PR-AUC (primary) + at least one of (Recall, F1, ROC-AUC).
5. Actionable Risk Insights
    - **Segment predictions into risk buckets (e.g., High/Medium/Low risk based on predicted probability).**
    - **Provide recommendations: inspection prioritization, alert thresholds, and expected tradeoffs.**

API Deployment

Create an API using FastAPI that:

- accepts a JSON input containing the required feature fields (based on your final model design), and
- Return the classified result and other things based on your style of design.

Expected outcomes

Your submission should demonstrate:

- A clean and reproducible preprocessing pipeline.
- Strong EDA with time-series specific insights(if applicable)/visualizations.
- Feature engineering suitable for the problem..
- Model training and evaluation using proper time-based validation.
- Clear model comparison and final model justification.
- Export of predictions to a file and reporting of metrics.
- A concise written summary with actionable findings and limitations.
- Functional API

DeepFabric

Deliverables

Please submit a **public Git repository** (with meaningful commit history) containing the following:

1. A well-documented Jupyter Notebook containing:

- Data Preprocessing
- Exploratory Data Analysis(EDA) (Visualizations Required)
- Feature Engineering
- Model Development
    - build at least one ML/statistical model (your choice)
- Model Evaluation
    - proper train/validation strategy (time-based split or backtesting)
- Fine - Tuning(if you do any)
- Alternative Model Comparisons
- Output File
    - Generate an output file (CSV or Excel) containing predictions on the test set.
- Metrics / accuracies
    - Report at least two metrics (choose appropriately):
        - Already defined above.
    - Include a brief justification for metric choice

2. A short report (not more than 2 pages) in README.md summarizing:

- Forecasting methodology
- Forecasting approach
- Key findings
- Recommendations
- Assumptions and limitations

3. API Deployment

- Provide complete documentation of the API deployment, including relevant screenshots. Attach the screenshots in the github repo.

DeepFabric

Additional instructions

1. Reproducibility

- Your repository should run on a fresh machine with clear setup steps.
- Include `requirements.txt` (or `environment.yml`).
- Avoid hard-coded local paths.

2. Time-series validation (must)

- Use time-based splitting (no random split).
- Clearly indicate train/validation/test periods.

3. Integrity & attribution

- You may use open-source libraries.
- If you use any external references (blogs, repos, notebooks), cite them in the README.
- Your submission must reflect your own understanding—be prepared to explain decisions in a follow-up discussion.

4. Submission checklist (before you submit)

- Notebook runs end-to-end
- Missing values handled (if any) and documented
- EDA contains relevant plots
- At least one baseline + one additional model
- Model comparison table included
- Predictions exported to CSV/Excel
- README summary ≤ 2 pages, clear and professional
- Public repo link shared

Software Tools Required for Assessment

- GitHub Account : Create a GitHub repository named UP-Analytics-2026-Assessment-<candidateName> and make it public.
- Visual Studio Code (VSCode) : Use VSCode as your primary development environment.
- Jupyter Notebooks
    - Preferred: Run notebooks inside VSCode
    - Alternatives (if needed): Google Colab or Azure Notebooks

DeepFabric

- MySQL Workbench
    - Required for database access, queries, and schema/data inspection (as applicable)
- [Optional] API Framework (if exposing your model as an endpoint)
    - FastAPI Setup

Timebox

- Expected time: **2 days** from the date of receiving this Assessment.

Additional Instructions

- Build from scratch (No prebuilt Models or API Connections are encouraged)
- Document AI usage and Packages and Models used
- Optional hosted demo appreciated.